Parsek Missions - Design Document

Design specification for the Missions subsystem: the abstraction layer above raw recordings, whole-mission looping, launch-window periodicity, and per-window interplanetary transfer re-aim.

Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to a timeline, rewind to earlier points, and see previously recorded missions play back as ghost vessels alongside new ones. This document specifies the Missions subsystem, which turns a tree of raw recordings into a named, selectable, loopable mission and replays it phase-locked to the live sky.

Status: shipped. Mission abstraction + whole-mission looping (PR #958), launch-window periodicity / phase-locking (PR #963), zero-drift reschedule, mission/group name link (PR #977), interplanetary transfer re-aim (PR #981 / #982), and destination-SOI arrival hold (PR #1024 / #1026 / #1030). The player-facing description (sections 1.2 and 9) is current to the v0.10.5 Missions window redesign (title bar with Log, Interact column, Advanced-only loop grid).

Out of scope: logistics supply runs and supply routes are a DOWNSTREAM CONSUMER of this subsystem, not part of it (see `docs/parsek-logistics-supply-routes-design.md`; logistics depends on Missions, never the reverse). The recording / tree data model itself (forks, merges, branch points, the optimizer) is owned by `docs/parsek-flight-recorder-design.md` and `docs/parsek-recording-finalization-design.md`. Map and Tracking-Station ghost rendering is owned by `docs/parsek-ghost-trajectory-rendering-design.md`.

Related docs: `docs/dev/design-mission-abstractions.md` (the abstraction-hierarchy design note this consolidates), `docs/dev/design-mission-periodicity.md` (the periodicity design note this consolidates), `docs/dev/done/design-mission-tree.md` (the earlier mission-tree exploration), `docs/dev/done/plans/reaim-interplanetary-transfers.md` (the re-aim plan), `docs/dev/done/plans/mission-periodicity-phases.md`.

---

1. Introduction

A recording is a raw building block: one bounded trajectory plus events, scoped to a recording tree. A single flown mission produces many of them, structured as a directed graph (forks at controlled separations, merges at docks). Nothing in that raw graph says "this contiguous set of recordings is one logical, loopable unit." The Missions subsystem builds that missing middle layer so the unit is a defined, player-configurable thing.

The headline feature it enables: **loop a whole mission as one continuous repeating flight**, not a stutter of independently relaunching ghosts. A multi-leg, multi-branch mission (ascend, transfer, land, return) replays exactly as it was flown, then wraps as a unit, on a clock that can be **phase-locked to the live solar system** so each replay lines up with the sky (the orbit sits over the launch site, an interplanetary transfer reaches its target).

This document covers:

- The abstraction hierarchy (recording → tree → spine → selection → Mission).
- The Mission data model and its serialization.
- The Missions UI window.
- Mission-level looping over a shared span clock.
- Launch-window periodicity (phase-locking the relaunch).
- Interplanetary transfer re-aim and destination-SOI arrival alignment (high level; the deep math lives in the re-aim plan).
- Invariants, logging, tests, and the open gaps.

It deliberately does NOT cover logistics (the supply-run / supply-route layer above Missions), the recording / optimizer internals it consumes, or the map-rendering internals it drives.

1.1 What the player sees

Situation	What happens
Commits a flown mission (a tree)	A default Mission (everything included) is auto-created, named after the tree's root group

Situation	What happens
Opens the Missions window (its first tab)	Each mission is a two-line bar (title, summary, `Log`, `Watch` over `Rewind`, a collapse caret under the index) over one row per physical vessel or EVA kerbal, indented by separation lineage, debris hidden
Unchecks a vessel or an interval (Advanced)	That part drops from the loop selection (start / end trim)
Clones a Mission (Advanced)	A cheap duplicate of the selection + settings (no recording data copied), to make a variant
Toggles Loop on a Mission (Advanced)	The whole selected subtree replays together over its span and relaunches on a cadence
Loops a celestial mission	The relaunch is phase-locked to the next launch window so the orbit / transfer lines up with the live sky
Loops an interplanetary mission	The heliocentric transfer is regenerated per launch window (re-aim) so it always intercepts the target; the arrival is held so the landing site recurs
Reads the period cell (Advanced)	A read-only value (`~6.4d`, or a range `~13d-19d` for a zero-drift schedule) when phase-locked or re-aimed, its basis (`Mun window`, `varies`, `Duna transfer`) in the hover; an editable interval plus a unit button otherwise

1.2 Worked example (the drop-pod mission)

A mothership stack (controller M) carries a drop pod (controller D). At separation the recording graph forks: one branch continues as M (deorbit, recover), the other as D (transit, land). Debris shed by either rides along its parent leg.

```
#    Missions and vessels                                          Interact
1    Drop pod        [Clone] [Warp to...]  [Log]                  [  Watch  ]
v    Kerbin . 41m . Jebediah Kerman . Landed
                     [Delete] Loop [x] every [30] [sec]            [ Rewind  ]
[x]  \- M   Launch -> Decoupled (D) -> Recovered
[x]     \- D   Decoupled -> Landed                                 [Fly][Seal]
```

The bar's first line is the title and the Advanced loop grid beside `Log`; its second line is the collapse caret under the index (`v` here: a down caret while the rows show, a right caret while they are hidden), the summary, `Delete` and the `Loop ... every ...` row. `Watch` and `Rewind` stack in the Interact column, one per line. Each vessel row carries an include checkbox (Advanced); unchecking the pod (D) drops its stretch from the loop selection and greys its row.

Turn Loop on with a period >= the span: one shared span clock sweeps `[earliest included start, latest included end]` and wraps as a unit. Each leg renders only while the clock is inside its own recorded window, so the multi-leg, multi-branch mission plays back exactly like the original flight, then restarts. Debris rides along.

---

2. Design Philosophy

1. **The Mission is a player-configured entity, not a machine-inferred one.** Earlier work tried to auto-detect loopable units from loop flags (`design-chain-auto-loop`); that detection is abandoned. The Mission selection IS the member set: no inference, no heuristics deciding what loops together.
2. **Read models are pure projections; recordings are never mutated.** Every derived structure (mission structure, through-line, composition, periodicity, loop units) is recomputed from the recording tree with no Unity calls, no shared mutable state, and no recording mutation. This makes them trivially unit-testable and keeps the immutable-recording rule intact.
3. **Boundaries fall between whole recordings, never mid-recording.** The atom is the post-optimizer recording. A selection is always a whole number of legs. The subsystem never operates on the recorder's raw in-flight buffer.
4. **Topology lives in one place (the tree DAG), expressed twice (Recordings tab and Missions tab).** Dock / undock are recorded entirely as `RecordingTree.BranchPoints`; the Missions window is a different VIEW of that same topology, not a second source of truth. No machine-generated Mission rows on dock.
5. **Looping reuses the span clock; periodicity only chooses WHEN.** Phase-locking and re-aim change the relaunch UT and (for interplanetary) regenerate the transfer geometry; they never change the recorded trajectory or the loop mechanics. Off / unsupported is byte-identical to plain looping.
6. **Fail closed to faithful.** Any unsupported geometry (cross-parent rendezvous, 2+-moon destinations, deep chains) leaves the mission on the faithful replay path, never half-applied. Periodicity that cannot align a given window still launches on cadence

and shows the un-aligned state.

7. **Observable from the log alone.** Every store mutation, loop-unit build, phase-lock decision, and re-aim engagement logs its inputs and outputs so a KSP.log read reconstructs what looped, when, and why.

---

3. Terminology

Term	Definition
Mission	A persisted, named selection over ONE recording tree, plus loop settings. Stored as the SET of excluded through-line heads + excluded interval keys, not as a frozen include list. `Mission.cs`
Mission tree	The committed branch topology a Mission projects over, scoped by `Recording.TreeId`. Always has at least one Mission.
Leg	One controlled (non-debris) recording, post-optimizer. The atom. Debris is excluded from the spine and rides along its parent at playback.
Node	A boundary point between two consecutive legs (including a path's start and end). NOT a recording.
Mission structure	The directed controlled-leg fork-tree (a DAG: forks split at controlled separations, same-tree Dock / Board merges join) derived from a tree. `MissionStructure.cs`
Through-line	The collapse of all legs of one continuous controlled vessel (env-split continuations + the vessel-continuation child at each branch) into one entry. `MissionThroughLine.cs`
Composition node / interval	A physical vessel during one structural interval (ends only when a controller separates), labelled "pod x1, probe x1, crew x3". The unit of interval-level start / end trim. `MissionComposition.cs`
Selection	The included set, DERIVED live from current topology as everything not excluded. Contiguous per path; choosing branches at each fork.
Loop unit	The span-clock descriptor a looping Mission compiles to: owner index + member indices + span + two cadences + optional schedules / re-aim plan. `LoopUnit` (`GhostPlaybackLogic.SpanClock.cs`)
Span clock	The single clock that sweeps a unit's whole span `[SpanStartUT, SpanEndUT]`, phased from `PhaseAnchorUT`, wrapping as a unit. `TryComputeSpanLoopUT` (`GhostPlaybackLogic.SpanClock.cs`)
Periodicity / P	The recurrence period at which the included config's celestial constraints all line up (best-fit within tolerance). The next faithful launch is the smallest `UT0 + k*P >= now`. `MissionPeriodicity.cs`
UT0	The recorded launch UT = the trimmed mission's span start (earliest included member start).
Constraint	A phase requirement one included segment imposes: `Rotation(B)` (a surface segment must sit over its ground spot and connect to its inertial orbit) or `Orbital(C)` (an SOI entry must reach body C where C will be).
Re-aim	For cross-parent (interplanetary) missions where faithful recurrence is effectively centuries away, replacing the recorded heliocentric transfer with a per-window Lambert-solved transfer that intercepts the target every synodic window. `Source/Parsek/Reaim/`
Arrival hold	Loop-clock dead time inserted at the heliocentric-to-capture boundary so the in-SOI replay (and the deorbit) starts later in live time, aligning the landing-site rotation phase. The inverse of a loiter cut.
Zero-drift schedule	A non-uniform relaunch schedule for drifting multi-constraint configs, anchored on the tightest constraint, so error never accumulates. `MissionPeriodicity.TryBuildRelaunchSchedule`

Design-concept to implementation-class mapping (names diverge):

Concept here	Class
Mission	`Mission` (`Mission.cs:13`)
Mission store	`MissionStore` (`MissionStore.cs:10`)
Mission structure walk	`MissionStructureBuilder.Build` (`MissionStructure.cs:122`)
Through-line collapse	`MissionThroughLineBuilder.Build` (`MissionThroughLine.cs:38`)

Concept here	Class
Interval composition	`MissionCompositionBuilder.Build` (`MissionComposition.cs:63`)
Mission → loop units adapter	`MissionLoopUnitBuilder.Build` (`MissionLoopUnitBuilder.cs:38`); single-selection entry point `MissionLoopUnitBuilder.TryBuildLoopUnitForSelection` (M-MIS-11 item 1, consumed by `RouteOrchestrator.ResolveLoopUnit`)
Span clock	`GhostPlaybackLogic.TryComputeSpanLoopUT` (`GhostPlaybackLogic.SpanClock.cs`)
Periodicity solve	`MissionPeriodicity` (`MissionPeriodicity.cs`)
Re-aim	`Source/Parsek/Reaim/*`
Name ↔ group sync	`MissionGroupLink` (`MissionGroupLink.cs`)
Missions window	`UI/MissionsWindowUI.cs`

---

4. Mental Model

4.1 The abstraction hierarchy (bottom-up)

```
6. Supply run / supply route   <- DEFERRED to logistics (downstream consumer)
   ------------------------------------------------------------------------
5. Mission        a saved, named selection + loop settings (the persisted entity)
4. Selection      a contiguous-per-path subset of legs; chooses branches at forks
3. Spine          one PATH through the graph from a start node to an end node
2. Mission tree   the directed graph for one played mission (forks + merges), scoped by TreeId
1. Recording      the post-optimizer atom; boundaries fall only between recordings
```

Layers 1 to 2 already exist in the recording data (topology in `RecordingTree.BranchPoints`, `IsDebris` discriminating spine legs from debris twigs). Layer 3 is reconstructable but not a first-class object. Layers 4 and 5 are what this subsystem adds: a persisted `Mission` object plus the Missions window. Layer 6 is logistics.

A single spine is one path. A selection may follow several forks at once, making it a SUBTREE of paths (each path independently contiguous) that loop together on one shared clock. The headline whole-mission case is the multi-path one.

4.2 The two clocks of a looped mission

```
                OverlapCadenceSeconds (true launch-to-launch)
                |     |     |     |        <- flight scene self-overlaps when this < span
launch instances v     v     v     v
                [=====mission span=====]   <- one instance sweeps the whole span
                ^
                PhaseAnchorUT (clamped to >= spanEnd: first-play floor)
                |
                CadenceSeconds (>= span: the single-instance clock for KSC / TS)
```

- `CadenceSeconds` (span-clock cadence): the loop period RAISED to at least the span, so a single span instance never truncates. The Space Center and Tracking Station have no overlap machinery and always render one span instance off this clock.
- `OverlapCadenceSeconds` (true launch cadence): the loop period NOT raised to the span. When shorter than the span the flight scene relaunches the mission before the prior instance finishes, so several staggered instances play at once (self-overlap), capped by `GhostPlayback.MaxOverlapMissionInstances`.

4.3 Periodicity overlay (chooses WHEN cycle 0 lands)

```
plain loop:        relaunch at LoopAnchorUT + k * cadence   (arbitrary phase)
phase-locked:      relaunch snapped to UT0 + k * P          (faithful window, P quantized into cadence)
zero-drift:        relaunch on a non-uniform schedule       (drifting multi-constraint, tightest anchor)
```

```
re-aim (cross-parent): relaunch every synodic window; transfer REGENERATED per window;
                       arrival HELD so the landing-site rotation phase recurs
```

---

5. The hierarchy in detail

5.1 Recording (the atom)

"Recording" means the recording AS IT EXISTS AFTER the optimizer has run: exactly what the Recordings window shows. Atom boundaries come from two provenances, both already applied by the time the abstraction reads them:

- Recorder-side commits at discrete gameplay events (launch, dock `OnPartCouple`, undock `OnVesselsUndocking`, EVA / board, scene exit / recovery, controlled decouple).
- Optimizer-side splits at environment / body transitions (atmosphere / altitude / SOI), filtered (surface grazes, brief bracketed runs under 120s, cohesive cross-body coasts, and boundary seams are suppressed).

Boundaries do NOT exist at a general vessel switch (the spine continues on a new same-tree recording) or at staging.

5.2 Mission tree (a directed graph)

Scoped by `TreeId`. A DIRECTED GRAPH, not a linear spine: lines FORK at controlled separations (Undock / EVA / JointBreak whose child is `IsDebris=false`) and JOIN at same-tree Dock / Board merges (two `ParentRecordingIds`). A dock to a FOREIGN vessel (another tree) is a single-parent branch point; that cross-tree link is reconstructed at playback via PID linking in `GhostChainWalker`, not as a two-parent edge. Debris (`IsDebris=true`) are TWIGS: never spine-eligible, they only ride along their parent leg and are not shown in the Missions UI.

5.3 Spine (a path)

A PATH from a start node to an end node, choosing which fork to follow at each separation. Reconstructed by following `RecordingTree.BranchPoints` across forks / merges / switches, grouping a run's optimizer env-split legs by shared `ChainId`, ordering by `StartUT`. `ChainId` does NOT thread the whole spine (it resets at vessel switches and scene exits); `TreeId` is the stable scope. The derivation handles trees with multiple disconnected roots (a restore-completed post-switch recording can have `ParentBranchPointId == null`, joinable only by `TreeId` + `StartUT`). Contiguity is causal / chronological with UT gaps allowed (a switch-and-coast leaves a real gap until first modification).

5.4 Selection (a contiguous-per-path subset)

A selection of legs defining what is in the mission. Along each path it is a contiguous interval; at each fork it chooses which branch(es) to include.

- Including a leg pulls in its debris twigs automatically (twigs never get their own toggle).
- Trim rule (both edges): the START edge is the first included leg (everything before is greyed, pre-start). The END edge is set by exclusion: excluding a leg drops that leg AND everything downstream of it (its sequence-successors and, if it is a fork leg, that whole branch). At a merge, dropping this path's parent drops the merged child from THIS path only; the co-parent's own Mission is unaffected.
- Forks may select MULTIPLE branches: one branch = a single-path mission; both = the whole-mission subtree.
- tree : selection is 1 : many, and selections may OVERLAP (share the trunk or any legs). The engine already renders many concurrent ghost instances of one recording, so this is free.
- PERSISTED as the set of EXCLUDED through-line head ids (coarse) plus excluded interval keys (finer start / end trim); the included set is DERIVED live. An interval key is a composition interval (`MissionCompositionNode.HeadLegId`: a leg id, `<head>/segN` for a later structural interval, `<key>@dockM` for a docked sub-interval), derived from the recordings alone, so it names the same stretch of the same recording for every consumer (render windows, loop units, routes, the Log). The Missions vessel rows only group these intervals for display (a ship's row may cross from one composition run into its own post-undock run), and the per-vessel checkbox writes exactly the keys of the intervals its row draws; how the rows are grouped never changes which interval a stored key names. Because the optimizer preserves the earliest segment's id on split and merge, an excluded head id keeps matching after a re-split / re-merge, so the selection survives topology churn (new sub-legs inside an included through-line auto-join and stay included).

5.5 Mission (the persisted entity)

A persisted, named object wrapping a selection: a `TreeId`, the excluded sets, loop on / off + period + time unit + anchor UT, a name, and an archived flag. Multiple Missions may target the same tree with different selections. CLONE duplicates the DEFINITION only (never recording data), so variants are cheap.

Invariant: every recorded tree always has at least one Mission. A default Mission (everything included) is auto-created when the tree is first recorded; DELETE is blocked on the last remaining Mission for a tree. The first Mission in list order is the original (never a clone); clones insert right after it.

---

6. Data Model

6.1 `Mission` (`Mission.cs:13`)

All instance fields public (the class is `internal sealed`).

```
Id: string                                - GUID; fresh on load if missing
TreeId: string                            - the recording tree this projects over
Name: string                              - display name (kept in sync with the root group name)
ExcludedThroughLineHeadIds: HashSet<string> - dropped through-lines (coarse selection)   (:18)
ExcludedIntervalKeys: HashSet<string>       - dropped composition intervals (finer trim)  (:25)
IncludedForeignDockLinkIds: HashSet<string> - included cross-tree dock links (M-MIS-8; foreign Dock/Board BranchPoint
GUIDs)
LoopPlayback: bool                        - loop on / off                                (:35)
LoopIntervalSeconds: double = UntouchedLoopIntervalSentinel - period in seconds          (:36)
LoopTimeUnit: LoopTimeUnit = Sec          - Sec / Min / Hour / Auto display unit          (:37)
LoopAnchorUT: double = NaN                - UT loop was last enabled at; span clock phases from this (:42)
Collapsed: bool                           - Missions tab view state: only the two-line bar draws
SelectionSchemaGeneration: int            - interval-key schema the selection was authored under
```

Ctors: default + `Mission(id, treeId, name)` (`:50`). `Clone(newId)` (`:60`) duplicates the definition only.

6.2 `MissionStore` (`MissionStore.cs:10`)

Static, survives scene changes. Holds `List<Mission> missions` and `SuppressLogging`.

- Lifecycle (run in this order on load): `PruneOrphans` (`:70`), `EnsureDefaultsForTrees` (`:42`, seeds default name from `tree.AutoGeneratedRootGroupName`), `ReconcileSelections` (`:101`, drops stale excluded ids after topology change, warns), `NormalizeOneLoopPerTree` (`:256`).
- Loop control: `SetLoopEnabled(target, on, currentUT)` (`:211`).
- Mutation: `Clone` (`:278`), `CanDelete` / `Delete` (`:301` / `:315`), `FindOriginalMission` (`:330`), `IsOriginalMission` (`:343`), `RenameMission` (`:352`).
- Persistence: `Save` / `Load` (`:364` / `:382`).

6.3 Derived read models (never serialized)

All pure, rebuilt each build:

- `MissionStructure` + `MissionStructureBuilder.Build` (`MissionStructure.cs:98`, `:122`) - the controlled-leg fork-tree DAG.
- `MissionThroughLine` / `MissionThroughLineView` + `MissionThroughLineBuilder.Build` (`MissionThroughLine.cs:14`, `:38`) - through-line collapse.
- `MissionCompositionNode` + `MissionCompositionBuilder.Build` (`MissionComposition.cs:28`, `:63`) - per-interval composition.

6.4 Loop-unit types (`GhostPlaybackLogic.SpanClock.cs`)

```
LoopUnit (internal readonly struct)
  OwnerIndex: int            - earliest-start member (camera / debris-parent representative)
  MemberIndices: int[]       - positional indices into RecordingStore.CommittedRecordings
  MemberWindow               - per-member trimmed [StartUT, EndUT]
  SpanStartUT / SpanEndUT    - [min trimmed start, max trimmed end]
  PhaseAnchorUT              - cycle-0 reference (clamped to >= SpanEndUT: first-play floor)
  CadenceSeconds             - span-clock cadence (>= span)
```

```
    OverlapCadenceSeconds      - true launch cadence (Auto = global auto-loop interval)
    relaunchSchedule           - optional zero-drift non-uniform schedule
    reaimPlan / reaimSchedule  - optional re-aim per-window transfer plan
    loiterCuts / arrivalHold   - optional re-aim launch-loiter compression + arrival alignment
    ArrivalHoldSeconds         - re-aim arrival hold amount inserted at the heliocentric->capture boundary (0 when none)
    ArrivalHoldAtUT            - recorded-span UT the hold is inserted at (NaN when none)
    ArrivalAlignPeriodSeconds  - per-loop destination-side align period: T_rot (landing), T_station
                                 (station rendezvous), or the M-MIS-6 config period T_config (2+-moon); NaN when none
    DescentMemberIndices       - committed indices of the re-aim looped-landing descent-set members
                                 (S4 arrival re-stitch); null when the descent trigger is not engaged

  LoopUnitSet                  - immutable per-frame snapshot, one unit per looping Mission;
                                 OwnerByIndex maps each committed index to exactly one unit; Empty = nothing loops
  LoopCut                      - one excised [start, len] span (loiter compression)
```

The INDEX CONTRACT is the seam tying scenes together: `OwnerIndex` / `MemberIndices` / `OwnerByIndex` are positional integer indices into `RecordingStore.CommittedRecordings`. Flight, KSC, and Tracking Station all key per-recording state on that same `int`, so one `LoopUnitSet` is valid in every scene.

6.5 Periodicity types (`MissionPeriodicity.cs`)

`ConstraintKind` (`:24`), `PhaseConstraint` (`:40`), `Support` enum (`:79` , e.g. `Supported` / `UnsupportedCrossParent` / `UnsupportedRendezvous`), `TransitedBodyRotationMode { Drop, Loose, Tight }` (`:106`), `ConstraintExtraction` (`:128`), `PeriodicitySolution` (`:154`), and the `IBodyInfo` seam (`:212`) with the FlightGlobals-backed `FlightGlobalsBodyInfo` . The seam keeps the solver pure and headless-testable.

6.6 Serialization

Mission state persists through `ParsekScenario` OnSave / OnLoad into the `.sfs` (lightweight; no sidecar). Derived read models are never serialized.

- Save (`ParsekScenario` → `MissionStore.Save`): purges the obsolete `missionHideArchived` / `missionWindowWidth` / `missionWindowHeight` keys, then writes one `MISSION` ConfigNode per mission via `Mission.Save` .
- Per-Mission keys (`Mission.Save`): `id` , `treeId` , `name` , `loopPlayback` , `loopIntervalSeconds` ("R" InvariantCulture), `loopTimeUnit` (enum name), `loopAnchorUT` ("R" InvariantCulture), `collapsed` (`Mission.Load` also reads an older `archived` key into it), `selectionSchemaGeneration` , repeated `excludedHead` , repeated `excludedInterval` , repeated `foreignDockLink` (M-MIS-8; SPARSE - written only when a cross-tree dock link is included).
- Load (`ParsekScenario.cs:2768` → `MissionStore.Load` `:382`): parses every `MISSION` node via `Mission.Load` (`:93` , every loop field defended with TryParse + field default; missing id → fresh GUID), then runs the four lifecycle passes in order.

---

7. Behavior

7.1 Creating and configuring a Mission

A default Mission (everything included) is auto-created when a tree is first committed, named after the tree's root group. In Advanced the player can clone it and trim the loop selection by unchecking vessels / intervals. Renaming a Mission renames the matching Recordings-tab group too (Section 8). The collapse caret hides a mission's rows under its bar; it changes nothing else.

7.2 Looping a Mission (the span clock)

When Loop is on, ALL included legs replay together over the span `[earliest included start, latest included end]` , relaunching on the cadence:

- Period >= span (or Auto resolves to >= span): ONE shared span clock sweeps the whole span and wraps as a unit. Each leg renders only while the clock is inside its own recorded window. A multi-leg, multi-branch mission plays back exactly like the original flight.
- Period < span: the mission OVERLAPS ITSELF, relaunching every `OverlapCadenceSeconds` so multiple staggered instances play concurrently (flight scene only; capped by `MaxOverlapMissionInstances`).

Debris rides along its parent leg via `ShouldSourceDebrisFromUnitSpan` (`GhostPlaybackEngine.cs:1765`). The mechanism reduces each member to a per-recording overlap loop over the EXISTING `UpdateOverlapPlayback` machinery, so the engine needed no new

playback path: `UpdateUnitMemberPlayback` (`GhostPlaybackEngine.cs:2119`) routes a member through the overlap path when the unit overlaps and the single span-clock instance otherwise.

7.3 The build pipeline

The host pushes a fresh `LoopUnitSet` once per frame: `MissionLoopUnitBuilder.Build(...)` (`MissionLoopUnitBuilder.cs:38`) → `GhostPlaybackEngine.SetLoopUnits(units)` (`GhostPlaybackEngine.cs:254`). Per looping Mission, `TryBuildMissionUnit` (`:111`):

1. Resolve the tree by `TreeId`.
2. Build read models: `MissionStructureBuilder.Build` → `MissionThroughLineBuilder.Build` → `MissionCompositionBuilder.Build`.
3. Compute trimmed member windows: `ComputeTrimmedMemberWindows` (`:650`) calls `MissionIntervalSelection.ComputeRenderWindows(compRoots, excludedIntervalKeys)` (`MissionIntervalSelection.cs:35`) for per-vessel `[StartUT, EndUT]`, intersects each member's recorded window, drops members entirely outside. This is the single source of truth shared by the span clock, the periodicity extractor, and the UI.
4. Sort members by trimmed start; span = `[min start, max end]`.
5. Derive the two cadences (Section 4.2).
6. Owner = earliest-start member.
7. Phase anchor: `LoopAnchorUT` (NaN → spanStart), then the FIRST-PLAY FLOOR clamps it to `>= spanEndUT` so a looped mission never relaunches before its first real play completes.
8. Periodicity / re-aim when `bodyInfo != null` (Section 7.4 / 7.5). Strict superset: no body-info or unsupported config keeps the raw cadences byte-identical.
9. Construct the `LoopUnit`.

A cheap `BuildSignature` (`:717`) gates the allocating `Build` so it only runs on input change.

7.4 Phase-locking to a launch window (same-parent periodicity)

For a celestial mission, an arbitrary relaunch time means the replay no longer matches the live sky: the recorded inertial orbit no longer sits over the launch site (the body has rotated), and a recorded transfer aims at where the target USED to be. Periodicity fixes WHEN the loop relaunches so the bodies are back in their recorded-launch configuration.

`MissionPeriodicity.ExtractConstraints` (`:283`) walks the trimmed member set and emits:

- `Rotation(B)` only when the set has BOTH a surface / atmospheric segment of B AND an inertial orbit of B (the ascent-to-orbit hand-off).
- `Orbital(C)` for every SOI-entry body. Direct-child orbital target = `Supported`; sibling / cross-parent = `UnsupportedCrossParent`; rendezvous / dock = `UnsupportedRendezvous`.

`Solve` (`:561`) picks the dominant constraint, locks `P = m * dominantPeriod` (joint best-fit `FindBestJointMultiple`, up to 16 multiples), and returns the next window `UT0 + k*P >= now`. In the builder (`MissionLoopUnitBuilder.cs:267`), when `ShouldPhaseLock` the anchor SNAPS to `NextWindowUT` and the cadence is quantized to a multiple of P (`QuantizeCadenceToMultipleOfP` `:621`). Tolerance is physics-derived (orbital: roughly `SOI_radius(C) / orbital_velocity(C)`; rotation: a small fraction of a degree); the residual is reported green when within tolerance, amber otherwise.

Drifting multi-constraint configs get a ZERO-DRIFT non-uniform schedule (`MissionPeriodicity.TryBuildRelaunchSchedule` `:1049`), anchored on the tightest-tolerance constraint so error never accumulates, throttled to the player's period. `TransitedBodyRotationMode` (Drop / Loose / Tight, UI labels Off / Loose / Precise) trades cadence against landing-handoff precision.

7.5 Interplanetary transfer re-aim (cross-parent)

For a cross-parent mission the exact recorded celestial configuration recurs only on the joint resonance of every transited body's period, which is effectively centuries away, so a pure replay-as-is loop would almost never launch. Re-aim removes that wall: the heliocentric transfer is REGENERATED per launch window with a Lambert solve so it always intercepts the target, while everything timing alone CAN make faithful (the launch-pad rotation lock, the recorded SOI-local arcs) stays as designed.

Engaged in `MissionLoopUnitBuilder.cs:329` only when not phase-locked. `ReaimClassifier.Classify` (`Reaim/ReaimClassifier.cs`) screens each member's SOI chain; supported single-hop interplanetary transfers get a synodic-window schedule from `ReaimWindowPlanner.Plan` (windows = `RecordedDepartureUT + k*synodic`, each re-solving Lambert for the target's actual position using the recorded time-of-flight). Supporting knobs:

- `ReaimLoiterCompressor.ComputeCuts` excises repeated parking orbits down to ~1 rev (launch side).
- `PadAlignLaunch` snaps the launch to the recorded body-rotation phase (whole sidereal day).
- `ArrivalHoldPlanner.ComputeArrivalHold` defers the in-SOI replay (loop-clock dead time at the heliocentric-to-capture boundary, the inverse of a loiter cut) so the destination's deorbit rotation phase recurs and the landing lands on the recorded site.

Per-window playback substitution happens at runtime via `ReaimPlaybackResolver.Shared` → `ReaimedTrajectory` (OrbitSegments only; delegates identity / PartEvents). The Lambert solve sits behind the `Reaim/ITransferSolver.cs` seam (`UvLambert.cs`). Everything fails closed to the faithful path. The deep derivation (the flexibility chain L0-L9, the arrival-hold / pre-landing-trim math, the destination-SOI arrival-UT solve and its feasibility envelope) lives in `docs/dev/design-mission-periodicity.md` and `docs/dev/done/plans/reaim-interplanetary-transfers.md`; this section is the overview.

7.6 Concurrent missions and self-overlap

Multiple Missions loop concurrently, at most one per tree. Enabling loop on a Mission clears it only on other same-tree Missions (same-tree variants share trunk legs, so they would collide on the single-owner `OwnerByIndex`). Different-tree Missions have disjoint committed indices and never collide. Self-overlap (period < span) is a flight-scene visual; KSC / TS always render one span instance.

---

8. Mission name and group-name sync

A tree's main mission name (Missions tab) and its root group name (Recordings tab) are the same abstraction shown in two places. `MissionGroupLink.RenameMissionGroup` (`MissionGroupLink.cs:57`) atomically renames the root group + the auto `/ Debris` and `/ Crew` subgroups + `Mission.Name`. Group names must be unique, so it collision-checks every target name up front and refuses the whole rename on any collision (the group side enforces uniqueness; the mission side does not). Both UI rename commit paths route through it; default names are seeded from `AutoGeneratedRootGroupName` so the two match from creation.

---

9. The Missions UI

`UI/MissionsWindowUI.cs` draws the Missions tab, the first tab of the `Parsek - Missions` window (`UI/RecordingsTableUI.cs` owns the chrome). The measured layout - every column, width, button and hover - is `docs/dev/design-gui-inventory.md` section 3.2; the Basic / Advanced split is `docs/dev/design-ui-basic-advanced.md` section 4.5. In short:

- Rows: one per physical vessel or EVA kerbal (`MissionVesselRowBuilder.Build`), indented by separation lineage, never by time; debris is not a row (it rides its parent). Another mission's vessel recorded into the flight (a partner flown to a dock, or its half after an undock) is a row beside the ship, not under it. At an undock where the docked pair kept the other mission's identity, the ship's row continues into the ship's own post-undock leg (`MissionVesselRowBuilder.ResolveUndockSides`, the same own-side rule as the Log's Undocked row), and the row's `Fly` / `Seal` is that post-undock flight's. In Advanced a vessel with more than one interval expands into its interval rows. Chapter header rows group rows, and `Docked partner:` rows name a dock with another mission's vessel, each with a `Go to` that opens that mission.
- Columns: `#`, `Missions and vessels`, `Start time`, `Start event`, `End event`, `End time`, and one `Interact` column that carries every per-row action. The header and every row kind lay out the same columns in both modes.
- The mission bar is two table rows. Line 1: index, bold title (double-click renames, through `MissionGroupLink`), then right-aligned (Advanced) `Clone` and `Warp to...` and `Log`; dates and events; `Watch`. Line 2: the collapse caret in the `#` column under the index (a clickable glyph, down while the rows show and right while they are hidden, hover `Collapse this mission` / `Expand this mission`), the muted summary (body, duration, crew, outcome, and in Advanced the amber `Next launch T- ...` of a looping mission), then (Advanced) `Delete` and `Loop [x] every [N] [unit]`; `Rewind` or `Forward` in the Interact column under `Watch`, at the same width. A route-bound tree reads `Looped by route` instead of the loop row.
- Loop toggle (`CommitMissionLoopToggle` → `MissionStore.SetLoopEnabled(mission, on, UT)`) is not drawn on a route-bound tree (`RouteTreeGuard` mutual exclusion: logistics is the downstream consumer).
- Period cell (`DrawMissionLoopPeriodCell`): an editable launch-to-launch interval plus a unit button cycling `sec` / `min` / `hr` / `auto` when not locked; a read-only value (`~6.4d`, a range for a zero-drift schedule) when phase-locked or re-aimed, with the basis (`Mun window`, `varies`, `<target> transfer`) and the locked-state sentence in the hover.
- Include checkboxes (Advanced): per vessel, per interval and per chapter, bound to `Mission.ExcludedIntervalKeys` (start / end trim, no cascade); the partner-journey link toggle binds `Mission.IncludedForeignDockLinkIds`.
- `Warp to...` (`DrawMissionWarpToWindowButton`, Advanced): fast-forwards to the mission's next relaunch after a confirm dialog; drawn in every Advanced state and greyed with its reason until the mission loops, has a built unit and a relaunch ahead, in a

warp-capable scene.
- The countdown on the summary targets the engine's ACTUAL next relaunch (off the real `LoopUnitSet`, not the next faithful P-window); a warned countdown ends in `(!)` with the warning in the summary's hover.

The display computes periodicity via `ComputeMissionPeriodicity` and the real unit via `GetLoopUnitSet` (built with `FlightGlobalsBodyInfo.Instance`, suppressing the pipeline log flags).

---

10. Edge Cases

1. **Empty / single-leg selection.** A mission with one included leg loops as a degenerate span (member window == span). Handled by the same path; no overlap.
2. **Member id not in CommittedRecordings.** The adapter skips ids not currently committed; the unit is built from whatever resolves. Out-of-range indices are never handed to the engine (consumers guard bounds but assume positional alignment).
3. **Excluded head id no longer resolves after topology change.** `ReconcileSelections` drops it and WARN-logs, so id churn fails loudly instead of silently re-including a dropped branch.
4. **Re-split / re-merge of an included through-line.** Safe: the optimizer preserves the earliest segment's id, so the excluded boundary keeps matching and new sub-legs auto-join (Section 5.4).
5. **A genuinely new branch (e.g. a re-fly supersede split).** Under the excluded-id model it defaults to INCLUDED. Accepted for v1 (Open Question 17.1).
6. **Two looping Missions on the same tree.** Forbidden by `SetLoopEnabled` (clears same-tree siblings) and re-enforced by `NormalizeOneLoopPerTree` on load (keeps the first per tree, warns on clearing extras); a defensive first-claimant guard in the builder covers a hand-edited save.
7. **Period < span.** Self-overlap in flight; one span instance in KSC / TS.
8. **Loop enabled mid-flight, before the span completes once.** The first-play floor clamps the anchor to `>= spanEndUT`, so the first relaunch waits for the first real play to finish.
9. **Phase-lock config that is over-constrained.** The best-fit window's residual exceeds tolerance; reported amber, still launched on cadence (fail closed to faithful).
10. **Cross-parent rendezvous / dock (`UnsupportedRendezvous`).** Not re-aimed; stays on the faithful path.
11. **2+-moon destination.** A 2+-moon RESONANT destination (Jool's inner three) is now SUPPORTED: it engages `ArrivalHoldPlanner.ComputeMultiMoonConfigHold` on the joint configuration period (M-MIS-6, Section 14.4). An incommensurate moon pack (Bop / Pol) finds no aligned window and ambers to the faithful replay. The retired 2+-moon reject was `DestinationConstraintExtractor`'s `MaxConstrainedMoons` gate, NOT `ReaimClassifier` (which only guards the heliocentric single-hop shape: the target must be a direct Sun child - Jool passes). A deep / multi-hop chain (target reached via another moon, or a second heliocentric leg) is still declined by `ReaimClassifier` and stays faithful (deferred, M-MIS-7).
12. **Re-aim window where the Lambert solve does not converge.** `BuildWindowSegments` returns null; that window degrades to the un-aligned faithful (Bug-2) render, surfaced as a distinct state, never garbage.
13. **Debris whose UT-covering leg is excluded while an overlapping leg is included.** It does not ride (it belongs to the leg it physically left). Acceptable v1; the adapter must not assume "parent leg" means the anchor parent.
14. **Delete on the last Mission for a tree.** Blocked (`CanDelete`), so a committed tree always keeps at least one Mission.
15. **Rename collides with an existing group name.** The whole rename is refused atomically (`MissionGroupLink`), no partial rename.
16. **Cold load with UT not yet ready.** Periodicity display and loop scheduling read `Planetarium.GetUniversalTime()` only from live scenes; the store load itself does no UT-dependent pruning.

---

11. What Doesn't Change

- The recording / tree data model, the optimizer, and `RecordingTree.BranchPoints`: consumed read-only. No mission code mutates a recording.
- The ghost playback engine's per-recording, non-unit playback path: a recording not in any active unit plays exactly as before.
- The Recordings window's own grouping (debris / crew / chain blocks): the Missions tab is an additional view, not a replacement.
- KSC / Tracking Station per-recording lifecycle and map presence: the span clock drives positioning, but the ghost / orbit-line lifecycle is the existing one.
- Save format for everything except the new `MISSION` nodes.

---

12. Out of Scope

- Supply runs and supply routes (logistics, the layer above Missions). Deferred to `docs/parsek-logistics-supply-routes-design.md`. Logistics consumes the Missions API read-only; the only mission-side coupling is the `RouteTreeGuard` mutual-exclusion check in the Loop toggle.
- True concurrent rendering of ONE recording on several Mission clocks (one ghost per Mission of the same recording). One-loop-per-tree sidesteps the single-owner conflict; deferred to logistics.
- Generalized destination-SOI alignment beyond the 2+-moon "mini star system" case: moons-of-planets and multi-hop chains (M-MIS-7).

---

13. Backward Compatibility

Pre-1.0 development rule: no legacy migration paths. The Mission nodes are additive `.sfs` content; a save without them loads with default Missions auto-created per tree by `EnsureDefaultsForTrees`. There is no recording-format or schema-generation change in this subsystem (it reads the existing tree topology). `Mission.Load` defends every field with TryParse + a field-default fallback, so a partially-written or older `MISSION` node loads to sane defaults rather than failing. Derived read models are recomputed every build, so no derived state can go stale across a version change.

---

14. Open Questions (deferred gaps)

These were settled-as-deferred at design time; most have since shipped (14.1 / 14.2 / 14.3 / 14.4 are RESOLVED below, leaving M-MIS-7 intra-SOI re-aim as the sole remaining gap). See `docs/dev/design-mission-abstractions.md` open questions 1-8 and `docs/dev/design-mission-periodicity.md` Phase 4 for the design discussion. The ordered, investigated completion plan for these gaps (with requirements and reuse mandates) is the **M-MIS milestone roadmap** at the top of `docs/dev/todo-and-known-bugs.md`: 14.1 → M-MIS-9, 14.2 → M-MIS-5, 14.3 → M-MIS-8, 14.4 → M-MIS-3 (inclined / eccentric targets) + M-MIS-6 (multi-moon window alignment) + M-MIS-7 (intra-SOI re-aim); looped rendezvous / station-dock alignment (not in the original list) is M-MIS-4.

14.1 New branches default to included (RESOLVED - M-MIS-9, 2026-06-12)

Original gap: a genuinely new branch added after a Mission was defined (most realistically a re-fly supersede split) defaults to INCLUDED under the excluded-id persistence model - acceptable for Missions themselves, but risky once logistics persists routes on top of Missions. Shipped for that risky consumer: routes freeze their backing selection to the creation-time member set (`Route` creation-time member freeze, PR #1127, including the positional-key hole fix), and route recovery credit is scoped to the creation-time source tree (M-MIS-9-R1, PR #1129), so a post-creation branch can no longer silently join a route's backing selection. Mission selections deliberately keep the default-included excluded-id model; plan: `docs/dev/done/plans/plan-mmis9-route-branch-freeze.md`.

14.2 Dock is not an interval boundary (RESOLVED - M-MIS-5 P1, 2026-07-04)

Shipped: `MissionCompositionBuilder.BuildNode` now emits an interval edge at every Dock / Board MERGE UT on the continuing line (gated on the run member's `OriginBranchPointType`), so the docked stretch is its own selectable sub-interval, keyed `<parentIntervalKey>@dockM` so structural `/segN` keys never renumber. The docked interval's label rebases to the merge leg's own start-captured combined composition (undercount fixed; structural peels on a rebased base subtract the departing leg's crew too), and pre-M-MIS-5 selections are upgraded once via `Mission.SelectionSchemaGeneration` + the `MissionStore.ReconcileSelections` @dock exclusion extension. Route render windows now end at the last DOCK (the realized route cycle re-aligns to DispatchInterval). Remaining logistics lift (accepting undock-to-undock shuttle runs) is M-MIS-5 P2a/P2b; plan: `docs/dev/done/plans/plan-mmis5-dock-interval-boundary.md`.

14.3 Cross-tree foreign dock (RESOLVED - M-MIS-8, 2026-07-07)

Shipped: a mission whose vessel was docked by a foreign tree's vessel can include that link's PARTNER JOURNEY (the docked stretch + its post-undock offshoot in the foreign tree) via an explicit "Partner journey" affordance in the Missions window. The Mission stays single-tree and persists only the included dock-link ids (`Mission.IncludedForeignDockLinkIds`, sparse `foreignDockLink` codec key - pre-existing missions round-trip byte-identically); the link and journey derive live via the same PID + launch-guid claim rule `GhostChainWalker` uses (`MissionCrossTreeDock`). Foreign members join the loop unit on ONE shared span clock; periodicity / re-aim fail closed to faithful for cross-tree units (two trees = two launches). Design note: `dev/design-mission-crosstree-dock.md`.

14.4 Destination-SOI alignment generalization (RESOLVED - M-MIS-6, 2026-07-08)

Shipped in 0.10.3 with `JoolConfigHoldInGameTest` as the automated merge gate: the 2+-moon "mini star system" case (Jool's resonant inner three) now aligns via a per-loop configuration hold, generalizing the earlier single-constrained-moon arrival hold. `DestinationConstraintExtractor` emits the full multi-moon set (all `MoonConfigs` plus the constrained moons' landing rotations in `MoonRotations`) instead of rejecting - the old `MaxConstrainedMoons` gate is retired - and `ArrivalHoldPlanner.ComputeMultiMoonConfigHold` owns the hold on the joint configuration period T_config (a whole number of anchor-moon periods, found by the joint recurrence scan) with the finite aligned horizon surfaced on engage. Incommensurate moon packs (Bop / Pol) find no aligned window and fail closed to the faithful replay with an AMBER arrival state (not a silent exclusion). Design note: `docs/dev/design-mission-multimoon-alignment.md`. Only moons-of-planets and deep multi-hop chains remain deferred (M-MIS-7).

---

15. Diagnostic Logging

Log format is fixed: `[Parsek][LEVEL][Subsystem] message`. All mission builders carry a `SuppressLogging` flag toggled by per-frame UI / route callers to avoid flood.

Tag	Owns
`[Mission]`	Store lifecycle (defaults, prune, reconcile, set-loop, normalize, clone / delete / rename), structure-build summary, loop-unit build summary + collision warns, group-link rename, Watch enter / exit
`[MissionPeriodicity]`	Phase-lock APPLIED (Info) / SKIPPED (VerboseRateLimited per-tree); constraint extract / solve summaries
`[Reaim]`	Re-aim ENGAGED / PAD-ALIGN / ARRIVAL HOLD (Info); eligible-but-invalid / not-re-aim (Verbose); per-loop hold (VerboseRateLimited, keyed on mission identity)
`[ReaimDiag]`	One-shot Verbose segment / cut dumps for save-reload diagnosis
`[RouteGuard]`	Loop toggle blocked when the tree is route-bound

Key event sites: `SetLoopEnabled` (Info, `MissionStore.cs:211`), `ReconcileSelections` / `NormalizeOneLoopPerTree` (Warn), structure-build summary (`MissionStructure.cs:211`), loop-unit collisions (`MissionLoopUnitBuilder.cs:71,90`), phase-lock decision (`:594,604`), re-aim engage / pad-align / arrival hold (`:494,510,533`). The engine counts inactive-unit frame skips under `MissionLoopUnitInactive` (`GhostPlaybackEngine.cs:743`). Convention: per-build / per-frame summaries are batch-counted and logged once (VerboseRateLimited), never per-item.

---

16. Test Plan

16.1 Unit tests (`Source/Parsek.Tests/`)

- **MissionStructureTests** (19) - leg fork-tree derivation: debris exclusion, sequence / branch edges, merges, roots, continuation flagging.
- **MissionCompositionTests** (16) - composition intervals, structural vs crew peels, label formatting, atom expansion, event-name mapping.
- **MissionLoopUnitBuilderTests** (35) - trimmed member windows, span / cadence derivation, owner selection, collisions, `ComputeTrimmedMemberWindows`, `BuildSignature`, `QuantizeCadenceToMultipleOfP`.
- **MissionStoreTests** (22) - defaults, prune, reconcile, `SetLoopEnabled` (one-loop-per-tree + concurrent cross-tree), normalize, clone / delete / find / rename.
- **MissionGroupLinkTests** (9) - main-mission ↔ root-group rename, subgroup cascade, collision refusal, clone-renames-alone, default-name seeding.
- **MissionSpanClockTests** (61) - `TryComputeSpanLoopUT` math: phase anchor, boundary epsilon, inter-cycle tail, loiter cuts, arrival hold, schedule path.
- **MissionPeriodicityTests** (53) - `ExtractConstraints` rules, `Solve`, dominant / anchor selection, joint best-fit, `NextWindow`, `CircularPhaseError`, transited-body modes.
- **MissionZeroDriftScheduleTests** (34) - `TryBuildRelaunchSchedule`, near-coincidence search, throttle / anchor behavior.
- **MissionsWindowPeriodicityDisplayTests** (37) - period / TTL display formatting, phase-locked vs scheduled vs re-aim labels, approx-period formatter.

- **MissionsWindowSortTests** (6) - mission-row sort keys + tiebreakers.

Re-aim carries its own non-`Mission*` test files (Lambert, window planner, loiter compressor, arrival hold, playback resolver).

16.2 In-game tests (`InGameTests/`)

- Span-clock single-ghost wrap, watch-follows-member across a member boundary, debris-follows-span (lifted from `design-chain-auto-loop`).
- Zero-drift reschedule in-game verification.
- Re-aim Duna One (s15) arrival-hold playtest (manual; the cross-parent landing alignment that cannot be CI-verified).

16.3 Coverage gate

`ParsekScenario` OnSave / OnLoad cannot be driven from xUnit (needs live Planetarium / Unity); hookup is covered by the source-text gate pattern, with the round-trip exercised in-game.

---

17. Implementation Status

Area	Scope	Status
Mission abstraction	Read models (structure / through-line / composition) + Missions window	Done (PR #958)
Whole-mission looping	Mission → LoopUnitSet adapter, span clock lifted, flight / KSC / TS parity, self-overlap, concurrent (one per tree)	Done
Periodicity	Constraint extract + solve, single + joint best-fit, phase-lock, TTL / faithful-period UI, first-play floor, "Warp to..."	Done (PR #963)
Zero-drift reschedule	Non-uniform schedule for drifting multi-constraint configs	Done
Name / group link	Mission name ↔ root group sync	Done (PR #977)
Interplanetary re-aim	Per-window Lambert transfer, loiter compression, pad-align, arrival hold	Done (PR #981 / #982, #1024 / #1026 / #1030)
Dock as interval boundary	Isolate a docked stretch for looping	Done (14.2, M-MIS-5 P1)
Cross-tree foreign dock	Loop a shared docked journey from the partner side	Done (14.3, M-MIS-8)
Looped rendezvous / landing + station dual arrival	Joint arrival hold via `DestinationArrivalSolver.SolveArrivalWindow` / `ArrivalHoldPlanner.ComputeJointArrivalHold`	Done (M-MIS-4)
Inclined / eccentric targets	Non-equatorial / eccentric destination arrival alignment	Done (M-MIS-3, code-complete)
Destination-SOI 2+-moon alignment	Joint config-hold for a resonant "mini star system" (Jool)	Done (14.4, M-MIS-6)
Destination-SOI multi-hop / moons-of-planets	Arrival alignment for deep / multi-hop chains	Deferred (M-MIS-7)

New source files: `Mission.cs`, `MissionStore.cs`, `MissionStructure.cs`, `MissionThroughLine.cs`, `MissionComposition.cs`, `MissionIntervalSelection.cs`, `MissionLoopUnitBuilder.cs`, `MissionPeriodicity.cs`, `MissionGroupLink.cs`, `MissionCrossTreeDock.cs`, `UI/MissionsWindowUI.cs`, `Reaim/*`, plus the new span-clock partial `GhostPlaybackLogic.SpanClock.cs` (loop-unit types + `TryComputeSpanLoopUT`) and the loop-unit consumption additions to `GhostPlaybackEngine.cs`.

---

18. Code Layout / Implementation Map

Concept	Code
Mission entity	`Mission.cs:13`

Concept	Code
Store + lifecycle + loop control	`MissionStore.cs` (`SetLoopEnabled` `:211`, `NormalizeOneLoopPerTree` `:256`)
Structure / through-line / composition	`MissionStructure.cs`, `MissionThroughLine.cs`, `MissionComposition.cs`
Interval start / end trim	`MissionIntervalSelection.cs` (`ComputeRenderWindows` `:35`)
Include / exclude selection cascade	`MissionSelection.cs`
Mission → loop units	`MissionLoopUnitBuilder.cs` (`Build` `:38`, `ComputeTrimmedMemberWindows` `:650`, `BuildSignature` `:717`)
Span clock + loop-unit types	`GhostPlaybackLogic.SpanClock.cs` (`LoopUnit`, `LoopUnitSet`, `LoopCut`, `TryComputeSpanLoopUT`)
Engine consumption	`GhostPlaybackEngine.cs` (`SetLoopUnits` `:254`, `UpdateUnitMemberPlayback` `:2119`)
Periodicity	`MissionPeriodicity.cs` (`ExtractConstraints` `:283`, `Solve` `:561`, `TryBuildRelaunchSchedule` `:1049`)
Re-aim	`Reaim/ReaimClassifier.cs`, `ReaimWindowPlanner.cs`, `ReaimLoiterCompressor.cs`, `ArrivalHoldPlanner.cs`, `ReaimPlaybackResolver.cs`, `ReaimedTrajectory.cs`, `UvLambert.cs`
Name / group sync	`MissionGroupLink.cs` (`RenameMissionGroup` `:57`)
UI	`UI/MissionsWindowUI.cs` (`DrawMissionsTabContent`, `DrawMissionHeader`, `ShowsLoopAuthoringControls`, `GetLoopUnitSet`), `MissionPresentation.cs`, `MissionVesselRows.cs`
Serialization	`Mission.Save` / `Load` (`Mission.cs:75` / `:93`), `MissionStore.Save` / `Load`, `ParsekScenario.cs:980` / `:2768`

References

- `design-mission-abstractions.md` - the abstraction-hierarchy design note (the 5-layer model, the Missions UI, span-clock integration, docking v1, open questions). Consolidated here.
- `design-mission-periodicity.md` - the periodicity design note (constraint model, P-solve, the re-aim flexibility chain L0-L9, arrival hold / pre-landing trim, destination-SOI arrival-UT solve, feasibility envelope). Consolidated here; consult it for the deep re-aim derivation.
- `done/design-mission-tree.md` - the earlier mission-tree exploration.
- `done/plans/reaim-interplanetary-transfers.md` - the re-aim implementation plan.
- `parsek-logistics-supply-routes-design.md` - the DOWNSTREAM consumer (supply runs / routes built on this subsystem). Logistics depends on Missions, not the reverse.
- `parsek-flight-recorder-design.md`, `parsek-recording-finalization-design.md` - the recording / tree data model this subsystem reads.

Consolidated 2026-06-09 from `design-mission-abstractions.md`, `design-mission-periodicity.md`, and the shipped implementation (PR #958 / #963 / #977 / #981 / #982 / #1024 / #1026 / #1030).